

MEG4 · MGFP SPECIFICATION

MEG4 OS — Guía oficial de integración MGFP

MeG4 Federation Protocol v1.0 · perfil commerce · documento técnico para cualquier servicio · Versión en español (ES)

MEG4 OS · Documento técnico oficial · Versión en español (ES)

MGFP — MeG4 Federation Protocol

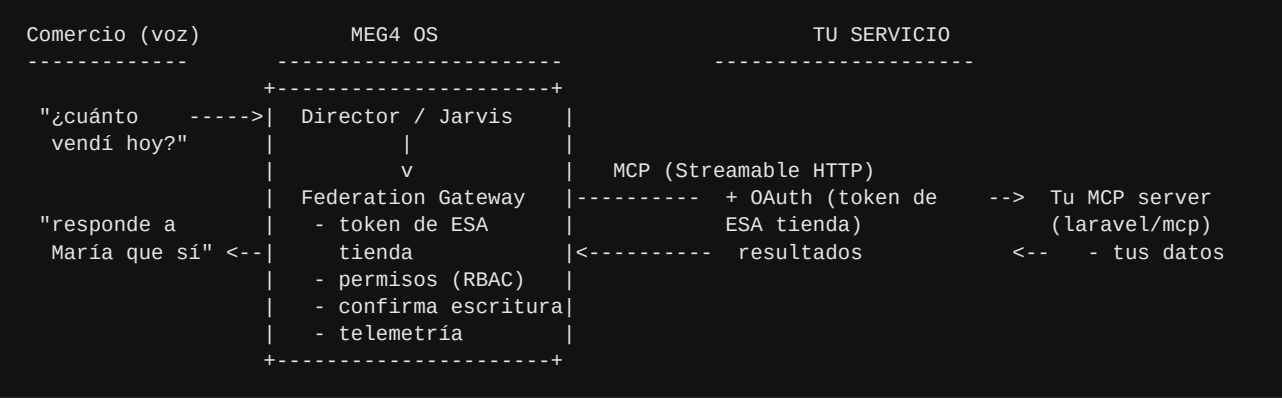
Versión 1.0 · Perfil `commerce` v1 · Documento en español (ES)

Alcance: esta es la **especificación técnica oficial** de MGFP para **cualquier** servicio que quiera integrarse con MEG4 OS. Usamos ``tu_servicio`` como nombre genérico; **Okastore** aparece solo como *ejemplo* concreto. Idioma de este documento: **español**. (*English version available on request.*)

Al integrarte, los comercios/usuarios que usan tu plataforma podrán **manejar todo su flujo de trabajo hablándole a Jarvis** — consultar productos, clientes, facturas y pedidos; leer y **responder mensajes como en WhatsApp**; crear facturas — **sin abrir tu app**.

La idea en una frase: tú expones un **servidor MCP estándar** (te sirve para *cualquier* asistente de IA, no solo MEG4). MEG4 le pone encima la capa de gobierno (OAuth por-comercio, permisos, confirmación de escrituras) y se lo entrega a Jarvis por voz. **No reinventamos ningún protocolo:** MGFP es MCP + un puñado de convenciones.

1. El modelo



- Tú corres un **MCP server genérico** en ``/mcp`` con las **tools canónicas** del perfil (§5) y lo proteges con un **token por tienda** (Sanctum; §4.3). El endpoint **no lleva nuestro nombre**: es tu MCP estándar, sirve a cualquier asistente. MGFP **no impone el path** — declaras tu `mcp_url` en el `service.json`.
- **MEG4** corre el *Federation Gateway*: descubre tus tools por `tools/list`, inyecta el **token del comercio correcto** en cada llamada, aplica permisos (leer es libre; **escribir requiere confirmación del dueño**), y expone todo a Jarvis namespaced como `tu_servicio_<tool>` (p.ej. `okastore_list_products`).

2. Lo que construyes (resumen)

1. Un **MCP server** genérico en `/mcp` sobre **Streamable HTTP** (con `laravel/mcp` son minutos — \$8). 2. **Solo el núcleo** del perfil `commerce` para arrancar: **7 tools de lectura** (catálogo + inbox + ventas). El resto (clientes, pedidos, facturas y las **escrituras**) es **opcional y progresivo** (\$5). MEG4 ofrece en Jarvis solo lo que expongas. 3. **Token por tienda** (Sanctum): el comercio genera su token y lo pega en MEG4 (\$4.3). OAuth click-to-connect queda opcional, no requisito. 4. Nos pasas tu **descriptor** (`service.json`, \$10) y listo.

Eso es todo. Tu lógica de negocio ya existe; solo la expones de forma estándar.

Plug-and-play (Laravel): instalá el SDK que te entregamos (git/zip; \$8) → `php artisan mgfp:install` → mapeás tus modelos en `config/mgfp.php` (o extendés `BaseStore`) → pegás el token. `mgfp:demo` y `mgfp:doctor` te lo prueban y validan. (El SDK es opcional: la spec de \$4–\$6 alcanza para integrarte en cualquier lenguaje.)

3. Conceptos

Término	Significado
Servicio	Tu plataforma (p.ej. <code>okastore</code>). Es el <code>serverInfo.name</code> de tu MCP.
Tienda / cuenta	Un comercio concreto dentro de tu plataforma. Un usuario de MEG4 puede vincular varias (\$7). Cada una tiene su token.
Perfil	El paquete de capacidades. Hoy: <code>commerce</code> v1 (\$5).
Tool de lectura	No modifica estado. <code>readOnlyHint: true</code> . Acceso libre.
Tool de escritura	Modifica estado (responder, facturar). No <code>readOnly</code> . Requiere permiso + confirmación + <code>idempotency_key</code> .

Tu MCP puede hacer más. MGFP es un **perfil** (un subconjunto contratado), no el dueño de tu server. Podés exponer en `/mcp` todas las tools que quieras (para tus propios agentes, ChatGPT, lo que sea): **MEG4 solo ve y usa las del perfil `commerce`**; el resto de tu server queda intacto e invisible para nosotros. Un solo MCP estándar sirve a todos.

4. Requisitos técnicos del MCP server

4.1 Transporte y endpoint

MCP remoto **Streamable HTTP** sobre **TLS** (HTTPS obligatorio). **Un solo endpoint genérico**, p.ej. `https://app.okastore.net/mcp`. El path es tuyo y MGFP **no lo impone** (nada de `/mcp/meg4`: es un MCP estándar para cualquier asistente). Lo declaras en tu `service.json` y MEG4 se conecta ahí.

4.2 Identidad y conformidad

En el `initialize`, tu `serverInfo` declara:

```
{
  "name": "okastore",
  "version": "1.0.0",
  "_meta": { "mgfp": { "version": "1.0", "profile": "commerce" } }
}
```

- `serverInfo.name` es tu ``service_id`` (cómo te registramos en MEG4).
- `_meta.mgfp` declara conformidad MGFP y el perfil.

4.3 Autenticación — token por tienda (default) u OAuth (opcional)

Default recomendado — ``api_key`` (token por tienda): el más fácil y seguro. El comercio genera **un token por tienda** (p.ej. **Laravel Sanctum**: hashado, con *abilities* `mgfp:read` / `mgfp:write`, revocable) y lo **pega una vez** en MEG4. Validarlo en tu MCP es **un guard** (`auth:sanctum`). No montas servidor OAuth. Es seguro: por-tienda, scopeado, revocable, sobre TLS. En tu `service.json`: `"auth": { "type": "api_key", "token_scheme": "Bearer", "whoami_url": "...", "scopes": [...] }`. El `whoami_url` (opcional) deja que MEG4 resuelva la **identidad de la tienda** (`account_id` / `store_name`) desde el token — clave para multi-tienda.

Opcional — ``oauth2`` (click-to-connect): authorization-code + PKCE (Passport). Mejor UX (el comercio aprueba en una pantalla) a cambio de más setup. Declara `authorization_endpoint`, `token_endpoint`, `client_id`, PKCE S256 y scopes.

Multi-tienda (importante en ambos modos): **un token por tienda**, y la identidad de la tienda resoluble (token con `mgfp:read/write` + `whoami`, o claims `account_id/store_id/store_name`). Así MEG4 enruta bien cuando el dueño tiene varias.

4.4 Anotaciones → permisos (clave)

MEG4 deriva el permiso de cada tool de sus **anotaciones MCP**:

Anotación	Significado	Permiso MEG4
<code>readOnlyHint: true</code>	no muta estado	<code>:read</code> — libre
(sin <code>readOnlyHint</code>)	muta estado	<code>:write</code> — requiere habilitación del dueño + confirmación por voz
<code>idempotentHint: true</code>	reintentar no duplica	obligatorio en escrituras, con <code>idempotency_key</code>
<code>destructiveHint: true</code>	puede borrar/sobrescribir	refuerza la confirmación

Marcar bien `readOnlyHint` es esencial: si olvidas marcarlo en una tool de lectura, MEG4 la tratará como escritura (default seguro) y pedirá confirmación.

4.5 Errores

Devuelve errores MCP estándar (`isError: true` con un bloque de texto explicativo, o error JSON-RPC). No inventes datos: si algo no existe, dilo. MEG4 narra el error al dueño con honestidad.

5. Catálogo de tools canónicas — perfil commerce v1

Exponé estas tools con **estos nombres exactos**. MEG4 las namespacea a `okastore_<tool>` (o `okastore_<alias>_<tool>` si el dueño tiene varias tiendas). Listados con paginación `cursor` (string) y `limit` (1–100, default 20).

Núcleo mínimo (arranca con esto): las 7 lecturas marcadas *núcleo* — `list_products`, `get_product`, `get_stock`, `unread_summary`, `list_threads`, `list_messages`, `analytics_summary`. Con solo eso ya estás en vivo. Lo demás (clientes, pedidos, facturas y las **escrituras**) es **opcional**: lo agregás cuando quieras y MEG4 lo detecta solo.

Catálogo e inventario

``list_products`` · lectura — *Lista o busca productos del catálogo.* { `filter?: string`, `cursor?: string`, `limit?: int` } → { `records: Product[]`, `next_cursor: string|null` }

``get_product`` · lectura — *Producto por id, con variantes y precios.* { `product_id: string` } → { `record: Product` }

``get_stock`` · lectura — *Stock disponible de un producto/variante.* { `product_id: string`, `variant?: string` } → { `product_id`, `variant`, `available: int` }

Clientes

``list_customers`` · lectura — *Lista o busca clientes.* { `filter?`, `cursor?`, `limit?` } → { `records: Customer[]`, `next_cursor` }

``get_customer`` · lectura — *Cliente por id, con historial resumido.* { `customer_id: string` } → { `record: Customer` }

Pedidos

``list_orders`` · lectura — *Lista pedidos (filtro por estado/fecha).* { `filter?`, `cursor?`, `limit?` } → { `records: Order[]`, `next_cursor` }

``get_order`` · lectura — *Pedido por id, con líneas, totales y estado.* { `order_id: string` } → { `record: Order` }

Facturas

``list_invoices`` · lectura — *Lista facturas emitidas.* { `filter?`, `cursor?`, `limit?` } → { `records: Invoice[]`, `next_cursor` }

``get_invoice`` · lectura — *Factura por id, con líneas, impuestos y total.* { `invoice_id: string` } → { `record: Invoice` }

``create_invoice`` · **escritura** · idempotente — *Crea una factura para un pedido (o líneas explícitas) y opcionalmente la envía.* { `order_id?: string`, `line_items?: LineItem[]`, `send_to_customer?: bool=true`, `idempotency_key: string` } → { `record: Invoice`, `sent: bool` }

Inbox (chats omnicanal)

``unread_summary`` · lectura — *Resumen compacto de no leídos (web/WhatsApp/IG). El payload con que Jarvis anuncia.* { `max_threads?: int=5` } → { `total_unread: int`, `top_threads: Thread[]`, `fetch_at: ISO8601` }

``list_threads`` · lectura — *Lista hilos (contactos/grupos).* { `channel?: "web"|"whatsapp"|"instagram"|"any"=any`, `unread_only?: bool=false`, `cursor?`, `limit?` } → { `records: Thread[]`, `next_cursor` }

``list_messages`` · lectura — *Mensajes de un hilo, recientes primero.* { `thread_id: string`, `limit?: int=30` } → { `records: Message[]` }

``reply_message`` · **escritura** · idempotente — *Responde a un hilo por su mismo canal (igual que en WhatsApp).* { `thread_id: string`, `text: string`, `idempotency_key: string` } → { `message_id: string`, `thread_id: string`, `status: "sent"|"failed"` }

Analítica

``analytics_summary`` · lectura — *Resumen de ventas del período.* { `period?: "today"|"yesterday"|"this_week"|"this_month"|"last_30d"=today` } → { `period`, `sales_count: int`, `revenue: number`, `top_products: [...]` }

Las formas `Product` / `Customer` / `Order` / `Invoice` / `Thread` / `Message` son libres (devuelve tus campos); incluye al menos un `id/*_id`, un nombre y los importes. Mientras más limpio el JSON, mejor narra Jarvis.

6. Ejemplos de protocolo

``tools/list`` (extracto) — anota `readOnlyHint`:

```
{ "tools": [
  { "name": "list_products",
    "description": "Lista o busca productos del catálogo.",
    "inputSchema": { "type": "object", "properties": {
      "filter": { "type": "string", "cursor": { "type": "string" },
      "limit": { "type": "integer", "minimum": 1, "maximum": 100, "default": 20 } } },
    "annotations": { "title": "Listar productos", "readOnlyHint": true } },

  { "name": "reply_message",
    "description": "Responde a una conversación por su mismo canal.",
    "inputSchema": { "type": "object",
      "properties": { "thread_id": { "type": "string", "text": { "type": "string" },
        "idempotency_key": { "type": "string" } } },
    "required": ["thread_id", "text", "idempotency_key"] },
    "annotations": { "title": "Responder mensaje", "readOnlyHint": false, "idempotentHint": true } }
]}
```

`tools/call` lectura:

```
// -> request
{ "name": "get_stock", "arguments": { "product_id": "p_botas_negras", "variant": "38" } }
// <- result
{ "structuredContent": { "product_id": "p_botas_negras", "variant": "38", "available": 3 } }
```

`tools/call` escritura (idempotente). MEG4 solo la invoca **después** de que el dueño confirma por voz:

```
// -> request
{ "name": "reply_message",
  "arguments": { "thread_id": "t_maria", "text": "¡Sí! Hay talla M, cuesta 25 $.",
    "idempotency_key": "mgfp-7f3a..." } }
// <- result
{ "structuredContent": { "message_id": "m_88", "thread_id": "t_maria", "status": "sent" } }
```

7. Multi-tienda

Un mismo dueño puede vincular **varias tiendas** de tu plataforma (p.ej. su tienda de ropa y la de zapatos). Para que funcione:

1. Emite un **token OAuth por tienda** (cada autorización = una tienda). 2. Haz resoluble la **identidad de la tienda** desde el token ([account_id/store_id](#)).

MEG4 se encarga del resto: le pone un **alias de voz** a cada tienda y, cuando hay más de una, cualifica los nombres ([okastore_ropa_list_products](#), [okastore_zapatos_list_products](#)) para que el dueño diga "¿cuánto vendí en la de ropa?". Tu server no necesita saber nada de esto: cada request ya trae el token de la tienda correcta.

8. Implementación en Laravel (SDK opcional)

El SDK es opcional. Para integrarte solo necesitas exponer el MCP server descrito en §4–§6 (en el lenguaje que sea). Para **Laravel** te damos un SDK que hace casi todo. Disponible como **repo git público**; se instala vía **composer VCS** (aún no en Packagist).

```
# 1) Declara el paquete que te entregamos (elige UNA):
composer config repositories.mgfp vcs https://github.com/meg4kz/meg4-mgfp-laravel.git # si te damos git
composer config repositories.mgfp path ./meg4-mgfp-laravel # si te damos el .zip (descomprimos)

# 2) Instálalo + dependencias
composer require meg4/mgfp-laravel:^1.0 laravel/mcp laravel/sanctum

# 3) Listo: configura, prueba y valida
php artisan mgfp:install # publica config/rutas e imprime tu service.json
php artisan mgfp:demo # lo prueba con datos falsos (sin tu BD)
php artisan mgfp:doctor # valida la conformidad (✓/✗)
```

Conectar tus datos = config, no código (para esquemas estándar). El SDK usa **EloquentAutoStore**, que mapea tus modelos desde **config/mgfp.php**:

```
'store_column' => 'store_id', // scoping multi-tienda
'capabilities' => ['list_products', 'get_product', 'get_stock',
                  'unread_summary', 'list_threads', 'list_messages', 'analytics_summary'],
'models' => [
    'products' => ['model' => \App\Models\Product::class,
                  'map' => ['id'=>'id', 'name'=>'name', 'price'=>'price'],
                  'search' => ['name'], 'stock_column' => 'stock'],
],
```

¿Lógica propia (inbox, facturar)? Extendé **BaseStore** (trae defaults seguros) y sobrescribí **solo** lo que tengas — lo no implementado/expuesto no aparece:

```
class OkastoreStore extends \Meg4\Mgfp\Support\BaseStore {
    public function replyMessage(string $storeId, string $threadId, string $text, string $idemp): array {
        $msg = Whatsapp::send($storeId, $threadId, $text, idempotencyKey: $idemp); // idempotente por $idemp
        return ['message_id' => $msg->id, 'thread_id' => $threadId, 'status' => 'sent'];
    }
}
// .env -> MGFP_STORE=\App\Mgfp\OkastoreStore
```

Auth por **token Sanctum** (ya en **routes/ai.php: Mcp::web('/mcp', ...)->middleware('auth:sanctum')**). El SDK trae las 15 tools, el **CommerceServer**, el auto-store, **BaseStore**, un **FakeStore** de demo y los comandos. Ver **federation/reference/laravel/README.md**.

8.1 ¿Ya tienes un MCP server propio?

MGFP es **compatible con un MCP preexistente** (con tus propios nombres de tools y, si quieres, para otros asistentes). Tres caminos:

1. **Exponé los nombres canónicos** (con el SDK o agregándolos a tu server). Ideal. 2. **Tools-alias finas**: agregás a tu MCP unas pocas tools con los nombres canónicos que delegan a tu lógica existente. 3. **Mapeo declarativo ('tool_map')** — **cero cambios de tu lado**. En el **service.json** mapeás canónico → tu tool real (y opcionalmente sus args). MEG4 expone los nombres canónicos al Director y traduce al invocar:

```
"tool_map": {
    "list_products": "getProducts",
    "reply_message": { "name": "sendChat", "args": { "thread_id": "chat_id", "text": "body" } }
}
```

Con eso, tu MCP queda intacto: MEG4 habla "canónico" y tu server recibe sus nombres/args de siempre. (Y como en §3, MEG4 **solo** toca las tools del perfil; el resto de tu MCP sigue siendo tuyo.)

9. Webhooks / notificaciones (opcional, recomendado)

Para que Jarvis avise **proactivamente** ("te escribió María", "entró un pedido"), puedes:

- emitir **notificaciones MCP** (`notifications/...`) en eventos `new_message` / `new_order`, o
- llamar un webhook firmado (HMAC) que MEG4 te indique al registrarte.

Es opcional: sin esto, Jarvis consulta `unread_summary` periódicamente.

10. Registro con MEG4 — `service.json`

Nos pasas (o publicas) este descriptor; MEG4 lo usa para alcanzarte:

```
{
  "mgfp_version": "1.0",
  "service_id": "okastore",
  "display_name": "Okastore",
  "profile": "commerce",
  "mcp_url": "https://app.okastore.net/mcp",
  "auth": {
    "type": "api_key",
    "token_scheme": "Bearer",
    "whoami_url": "https://app.okastore.net/mcp/whoami",
    "instructions": "Genera un token de tienda en Ajustes -> Integraciones -> MEG4.",
    "scopes": ["okastore:read", "okastore:write"]
  },
  "branding": { "voice_name": "Okastore", "locale": "es-VE" }
}
```

¿OAuth click-to-connect? Reemplaza el bloque `auth` por `"auth": { "type": "oauth2", "authorization_endpoint": "...", "token_endpoint": "...", "client_id": "...", "redirect_uri": "...", "use_pkce": true, "scopes": [...] }`.

11. Checklist de conformidad MGFP v1

- ■ MCP server genérico en `/mcp` sobre **Streamable HTTP + TLS** (sin `/meg4`).
 - ■ `serverInfo.name` = tu `service_id`; `_meta.mgfp` = `{version, profile}`.
 - ■ **Token por tienda** (Sanctum) — o OAuth opcional —, **identidad de tienda resoluble** (whoami/claims).
 - ■ Al menos el **núcleo** (7 lecturas) con los **nombres exactos**; el resto opcional.
 - ■ `readOnlyHint` correcto en lecturas; escrituras con `idempotentHint` + `idempotency_key`.
 - ■ Paginación `cursor/limit` en los listados.
 - ■ Errores honestos (`isError`), sin inventar datos.
 - ■ (Opcional) notificaciones/webhooks de `new_message` / `new_order`.
 - ■ `service.json` entregado a MEG4 (`mgfp:install` te lo imprime).
-

12. Seguridad y privacidad

- **Aislamiento por tienda:** un token solo abre su tienda; MEG4 nunca cruza datos entre comercios.
- **Escritura gobernada:** responder/facturar exige que el dueño habilite escritura y confirme cada acción por voz; MEG4 agrega `idempotency_key` para que un reintento no duplique.
- **Mínimo privilegio:** pide solo los scopes que uses; el dueño puede **revocar** el acceso cuando quiera y MEG4 lo pierde al instante.
- **Sin fugas:** MEG4 nunca registra el valor de tokens ni PII; la telemetría audita *quién pidió qué*, no el contenido. Los medios pesados viajan como referencias, no como blobs.

MGFP v1.0 — MEG4 OS. Familia de protocolos: MGD_P (Director) · MGB_P (Bridge) · MGPP (Phone) · MGFP (Federation). Dudas de integración: el equipo de MEG4 OS.